

Performance mining for batch processing using the performance spectrum

Citation for published version (APA):

Klijn, E. L., & Fahland, D. (2019). Performance mining for batch processing using the performance spectrum. In C. Di Francescomarino, R. Dijkman, & U. Zdun (Eds.), *Business Process Management Workshops - BPM 2019 International Workshops, Revised Selected Papers* (pp. 172-185). (Lecture Notes in Business Information Processing; Vol. 362 LNBIP). Springer. https://doi.org/10.1007/978-3-030-37453-2_15

Document license:

CC BY-SA

DOI:

[10.1007/978-3-030-37453-2_15](https://doi.org/10.1007/978-3-030-37453-2_15)

Document status and date:

Published: 01/01/2019

Document Version:

Accepted manuscript including changes made at the peer-review stage

Please check the document version of this publication:

- A submitted manuscript is the version of the article upon submission and before peer-review. There can be important differences between the submitted version and the official published version of record. People interested in the research are advised to contact the author for the final version of the publication, or visit the DOI to the publisher's website.
- The final author version and the galley proof are versions of the publication after peer review.
- The final published version features the final layout of the paper including the volume, issue and page numbers.

[Link to publication](#)

General rights

Copyright and moral rights for the publications made accessible in the public portal are retained by the authors and/or other copyright owners and it is a condition of accessing publications that users recognise and abide by the legal requirements associated with these rights.

- Users may download and print one copy of any publication from the public portal for the purpose of private study or research.
- You may not further distribute the material or use it for any profit-making activity or commercial gain
- You may freely distribute the URL identifying the publication in the public portal.

If the publication is distributed under the terms of Article 25fa of the Dutch Copyright Act, indicated by the "Taverne" license above, please follow below link for the End User Agreement:

www.tue.nl/taverne

Take down policy

If you believe that this document breaches copyright please contact us at:

openaccess@tue.nl

providing details and we will investigate your claim.

Performance Mining for Batch Processing Using the Performance Spectrum

Eva L. Klijn and Dirk Fahland^[0000–0002–1993–9363]

Eindhoven University of Technology, The Netherlands
e.l.klijn@student.tue.nl, d.fahland@tue.nl

Abstract. Performance analysis from process event logs is a central element of business process management and improvement. Established performance analysis techniques aggregate time-stamped event data to identify bottlenecks or to visualize process performance indicators over time. These aggregation-based techniques are not able to detect and quantify the performance of time-dependent performance patterns such as batches. In this paper, we propose a first technique for *mining* performance features from the recently introduced *performance spectrum*. We present an algorithm to detect batches from event logs even in case of batches overlapping with non-batched cases, and we propose several measures to quantify batching performance. Our analysis of public real-life event logs shows that we can detect batches reliably, batching performance differs significantly across processes, across activities within a process, and our technique even allows to detect effective changes to batching policies regarding consistency of processing.

Keywords: process mining · performance mining · batch processing · performance spectrum.

1 Introduction

Analyzing process performance based on event data is a central activity in business process intelligence and process management. Its aim is to identify performance characteristics and problems such as bottlenecks, unusual variations in performance, or permanent drifts and changes in performance. In the following we focus on time-based performance analysis. Standard techniques to performance analysis are to implement key performance indicators (KPIs) in the running system to measure throughput times, waiting times, etc. [5, Ch.7], or to derive performance information from event logs [5, Ch.10] which is then *aggregated*, e.g., average waiting times of all cases at a task, and then visualized in charts or by annotation of process models [1].

The recently introduced *performance spectrum* [4] showed that all processes exhibit non-stationary performance leading to local performance variations, drifts in performance, and performance patterns. Even the same process step may exhibit two or more different discernible performance characteristics. For example, *batch processing* allows grouping several cases arriving at a task into a queue and to process them together in a short period [13,8]. While batching is an effective technique to improve process performance [12] the performance spectrum in Fig. 2 shows that batch performance at one task may vary greatly over time and batching and non-batching behavior may

overlap. Aggregation-based performance techniques such as KPIs are not able to discern and quantify local performance characteristics of this kind.

In this paper, we consider the problem of *mining performance characteristics* directly from the performance spectrum. Specifically focusing on batching, our aim is to (1) detect and discern batching from non-batching behavior in any process step, allowing (2) to specifically quantify and compare batching performance and non-batching performance. We contribute an *algorithm to detect batches* from event logs – even when overlapping with non-batching behavior; our only parameter is the minimum batch size. We introduce several *measures to quantify batches* such as size, waiting time in/outside a batch, arrival time in/outside a batch, and batching frequency. We implemented the algorithm and the measures and applied them on several public real-life event logs. We found significant batching in 4 of the event logs; batching is in most cases overlaid with non-batching behavior; the batching characteristics of different process steps are significantly different and subject to large variations over time our technique allows to detect.

Next, we first discuss related work on analyzing batch processing in Sect. 2. We then recall the various types of batch processing and how they are described in the performance spectrum in Sect. 3. We define the batching measures in Sect. 4 and the detection algorithm in Sect. 5 which we evaluate in Sect. 6. We conclude in Sect. 7.

2 Related Work

Batch processing has been studied in several domains. In operations management it is used to solve supply chain scheduling problems [15], order picking problems [6] and machine scheduling problems [2], with the aim of reducing costs in holding, inventory, and machine setup as a trade-off to increased delivery and service times.

In business process management, batch processing received increased attention over the past five years. Pufahl and Weske [13] proposed to explicitly model batch activities, which allows to evaluate batch activity performance [12]. Thereby, batch performance is not just activity-dependent but subject to additional process perspectives such as resource involvement [14], which is confirmed by the performance spectrum [4].

The problem of discovering batch process models from event logs has been explored by Wen et al. [17]. The proposed algorithm requires a prior knowledge of where batching takes place, which has to be provided in a pre-processing step. Nakatumba explores the problem of identifying batch processing from a resource perspective [11] by assuming resources work in batches and discovering batching moments from resource availability; the technique cannot measure detailed characteristics of batching behavior. Martin et al. [10] distinguishes and formalizes three types of batch processing by analyzing a matrix of resource-activity pairs for “footprints” corresponding to the three batching types. By calculating metrics, the influence of the batching behavior on the process performance can be derived. The technique was extended to discover batch activation rules from event logs [8] describing for example the time duration or number of cases required to cause processing of a batch. The work also shows that correct batch activation rules aid in more precise performance prediction (in simulation models).

This research elaborates on [10,8] in several ways. We adopt the three types of batch processing proposed and lift them to the performance spectrum, and we propose several

batch measures to quantify batch characteristics more precisely. We differ by detecting batching exclusively from the control-flow perspective (not requiring resource information or any other a priori input) by transforming event log data into the performance spectrum [4]. Further, our method specifically preserves the time-characteristics of all cases in batch detection allowing to quantify the detected batches.

Batching behavior and performance is also relevant in queue mining from event logs [16]. Our analysis of batching behavior between two subsequent tasks can also be seen as analyzing the batching behavior in a queue; however we do not consider other parameters considered in queue mining [16].

3 Batch processing and performance

In this section, we introduce the types of batch processing and the corresponding pattern in the performance spectrum. We then combine these concepts into a definition of a batch processing pattern, based on control-flow event logs only.

The processing of cases in a simultaneous manner or in batches, often occurs when they share properties or are processed periodically. These structural and behavioral properties of the process and its instances also influence the way cases are batched. Martin et al. [9] distinguishes three types of batching. *Simultaneous*: Cases are executed by the same resource and have an identical timestamp for the start and completion of the processing step, as illustrated in Fig. 1 (left). *Sequential*: Cases are executed by the same resource and the completion timestamp of each case must (roughly) correspond to the start timestamp of the case that follows, as illustrated in Fig. 1 (middle). *Concurrent*: Cases are executed by the same resource and the start timestamp of a case must be earlier than the complete timestamp of the preceding case, meaning cases should partially overlap in time. This is illustrated in Fig. 1 (right).

The *performance spectrum* [4] visualizes process behavior over time as illustrated in Fig. 2. It describes how cases transition from activity a to activity b over time t , also called the *segment* (a, b) . Whenever in a case a is directly followed by b , we observe an *occurrence* of this segment taking place from time t_a (moment of occurrence of a) to time t_b . Each occurrence of a segment is plotted as a line from (t_a, y_a) to (t_b, y_b) .

The arrangement of the lines in Fig. 2 forms distinct patterns which can be classified by a taxonomy [4]. This taxonomy defines batching as FIFO behavior where batching occurs at either at the preceding step a , the succeeding step b or both. We will focus on the case where batching occurs at the succeeding step b , called *batch(e)* [4]. On the performance spectrum, this will show up as lines starting at various points on y_a and converging to a more or less single point in time on y_b , as can also be identified in Fig. 2.

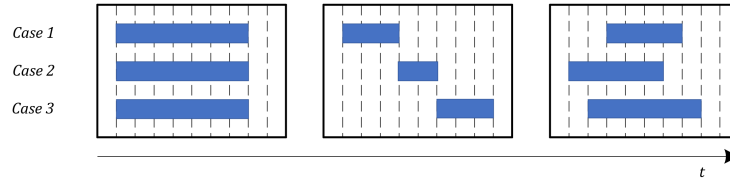


Fig. 1. Schematic representation illustrating three types of batching: simultaneous (left), sequential (middle) and concurrent (right)

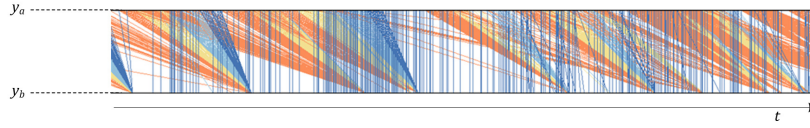


Fig. 2. Detailed performance spectrum example of single process segment of Road Traffic Fine Management log



Fig. 3. Schematic representations of batch processing types of Fig. 1 in the performance spectrum: simultaneous (left), sequential (middle) and concurrent (right)

To detect *batch(e)* from a process segment, the type of batch processing applied in the succeeding processing step must be taken into account. Also, a process segment only holds information regarding the start and end timestamp of cases within that segment, meaning that of each adjacent processing step, only a single timestamp is known. Thus, to distinguish the different types of batch processing for the succeeding processing step, we are limited to the distribution of timestamps at b . Figure 3 illustrates how the different types of batch processing of Fig. 1 and the corresponding distributions of timestamps at b are represented in the performance spectrum.

We see that simultaneous batch processing, having identical start and end timestamps for all cases, clearly distinguishes itself from the other types. Sequential and concurrent batch processing both have distinct start and end timestamps between cases, of which the distribution is dependent on case or processing step characteristics. Being that of each processing step either the start timestamp or the complete timestamp is known within a segment, no distinction can be made between sequential and concurrent batch processing. In the following, we therefore refer to the latter two types as *disjoint* batch processing to distinguish them from *simultaneous* batching processing.

Activity life-cycle information has limited influence on the batch patterns. In most cases, events are recorded using their start timestamp, complete timestamp or both, meaning that a segment will either be between two start events, two complete events or between the complete event of the preceding activity and the start event of the succeeding activity. When considering Fig. 3 once more, we see that for each of the batch processing types, the distribution of start timestamps is similar to the distribution of end timestamps, implying that different procedures of event log recording will not influence the process of detecting batches.

We now project the possible *batch(e)* patterns w.r.t. the batch processing types on the performance spectrum, illustrated in Fig. 4. Simultaneous batch processing shows non-crossing lines, all ending at the same time-point on y_b , illustrated in Fig. 4 (left). Disjoint batch processing shows lines ending at distinct time-points on y_b , either non-crossing (FIFO-ordered) or crossing (unordered), illustrated in Fig. 4, middle and right, respectively. For this paper, we decide to conform to the FIFO constraint of *batch(e)* as it is defined in [4] and will therefore discard the latter case.

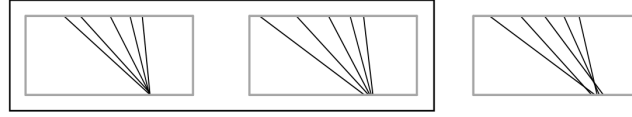


Fig. 4. Schematic representations of batching on end pattern: simultaneous (left), disjoint FIFO ordered (middle) and disjoint unordered (right)

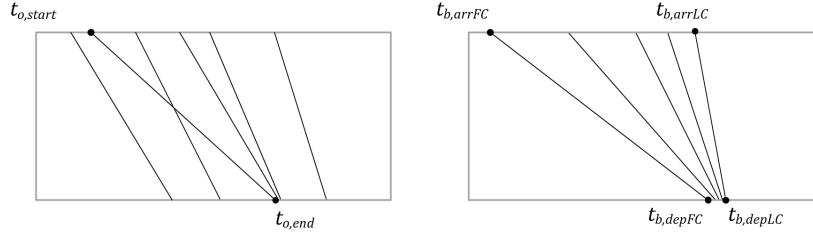


Fig. 5. Performance spectrum visualization of observation (left) and batch (right) time parameters

As a result, we define *batching on end* as follows: *Multiple cases are batched on end when the next processing step handles cases either (1) simultaneously, or (2) in the order they arrived in and in a very short period of time.*

4 Measuring batch performance in the performance spectrum

This section discusses several measures for quantifying the performance of batch processing for a given set of batches; Sect. 5 then describes the procedure to identify batches from the performance spectrum. We will first present the three main objects embodied by the algorithm along with their parameters, since the batch processing measures will be derived thereof.

Segment. A segment $s = (x, y)$ is defined by a pair of activities x and y and a start $t_{s,start}$ and end $t_{s,end}$ time for the batching analysis.

Observation. An observation $o = (o_{id}, t_{o,start}, t_{o,end})$ in segment (x, y) describes that the case o_{id} had activities $\dots, a, b, \dots$ follow each other with x and y occurring at $t_{o,start}$ and $t_{o,end}$, respectively, as illustrated in Fig. 5 (left).

Batch. A batch $b = o_1, \dots, o_k$ is a set of observations in (x, y) where the $t_{o_i,end}$ end timestamps of all observations in b are equal (simultaneous batch) or more or less equal (disjoint batch). In the latter case, observations within b must also satisfy the FIFO rule, meaning that if $t_{o_i,end}$ is smaller than $t_{o_j,end}$, $t_{o_i,start}$ must also be smaller than $t_{o_j,start}$. The interest lies with finding all batches in a segment (x, y) , i.e., multiple sets $b_1, \dots, b_m$, of a minimum batch size $k_{min} \leq |b|$.

In the following, we define the measures that are derived of the objects parameters defined in the previous, both for batch- and segment-level perspectives.

One batch. Each batch has a first case arrival time $t_{b,arrFC}$ and a last case arrival time $t_{b,arrLC}$, derived from $t_{o,start}$ of the first observation and $t_{o,start}$ from the last observation in b . Additionally, each batch also has a first case departure time $t_{b,depFC}$ and a last case departure time $t_{b,depLC}$, derived from $t_{o,end}$ of the first observation and $t_{o,end}$ of the

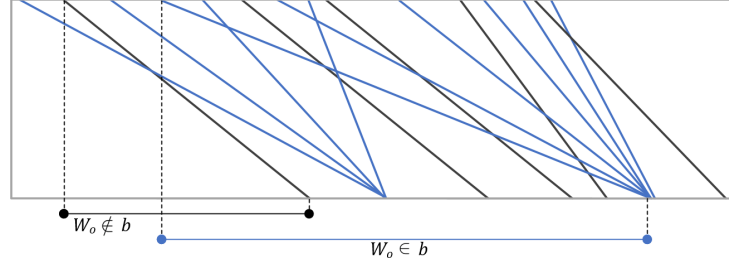


Fig. 6. Illustration of waiting time measures on the performance spectrum

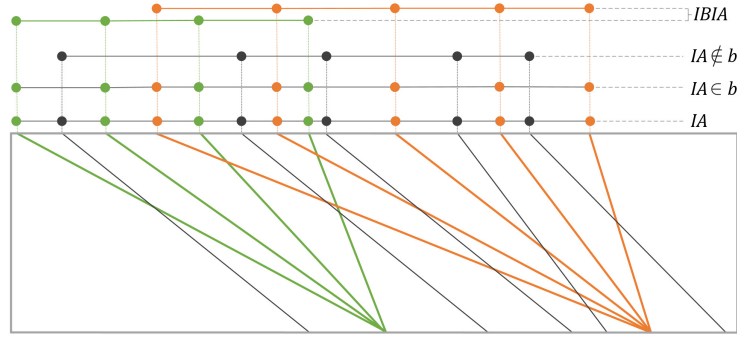


Fig. 7. Illustration of interarrival time measures on the performance spectrum

last observation in b . In the case of a simultaneous batch, $t_{b,depFC}$ and $t_{b,depLC}$ are equal. Fig. 5 (right) shows an illustration of these time parameters.

Case waiting time. We define W_o as the case waiting time or the time a case is pending before the next processing step. On batch-level, we will also define $W_{i,min}$ and $W_{i,max}$, expressing the minimum and maximum W_o of observations within batch i , respectively. This will give insights in the batching time-window used. On segment-level, we additionally define $W_o \in b$ as the waiting time of a batched case and $W_o \notin b$ as the waiting time of a non-batched case. W_o can directly be derived from the performance spectrum, as illustrated in Fig. 6.

Intra-batch case interarrival time. We define $IBIA$ as the intra-batch case interarrival time, expressing the amount of time that lapses between the arrival of two successive cases within a batch. Each batch consists of k observations, each observation having a time of arrival $t_{o,start}$ within that segment. From these times, $k - 1$ intra-batch interarrival times can be derived, as illustrated in Fig. 7. From a batch-level perspective, solely the $IBIA$ measures of either the green or orange highlighted batch would be taken into account, whereas from a segment level perspective, the $IBIA$ measures of both batches would be taken into account.

Case interarrival time. Where $IBIA$ measures arrival in a batch, the case interarrival time IA measures the time between two cases considering *all* segment observations. We additionally define $IA \in b$ and $IA \notin b$, expressing the interarrival time of batched cases (independent of which batch it belongs to) and non-batched cases, respectively. Figure 7 illustrates these measures and their difference to $IBIA$.

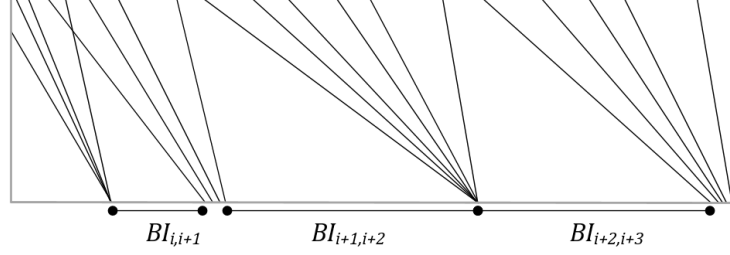


Fig. 8. Illustration of batch interval BI measure on the performance spectrum

Batch interval. We define the batch interval BI as the time lapse between two successive moments or periods of batching, as illustrated in Fig. 8. A moment or a period of batching from a segment perspective means the moment or period in which cases are taken into processing. When considering the batch parameters, this moment or period ranges from $t_{b,depFC}$ to $t_{b,depLC}$, where in the case of a simultaneous batch, the values of these parameters are equal. Intervals between batches are calculated on segment-level and, given there are m batches in a segment and a segment contains at least one batch $m \geq 1$, $m - 1$ batch intervals can be derived. The duration of a batch interval between batch i and its successive batch is calculated as follows: $BI_i = t_{b_{i+1},depFC} - t_{b_i,depLC}$.

Batch size. The batch size k expresses the amount of observations in batch b .

Batching frequency. We define BF as the batching frequency or percentage, expressing the fraction of observations that are part of a batch out of the total amount of observations within a segment. For a segment containing n observations and m batches, each with k_m observations, the batching frequency is calculated as follows: $BF = \sum_{i=1}^m k_i / n$

5 Detecting batches in the performance spectrum

To evaluate the presence of batching and provide performance insights, we propose the following Batch Detection Algorithm.

As input, we use the data from the performance spectrum miner [3], which is represented as a table with columns for s , o_{id} , $t_{o,start}$ and W_o . We first transform this data by (1) calculating $t_{o,end}$, (2) converting data to lists of observations per segment, containing columns for o_{id} , $t_{o,start}$, W_o and $t_{o,end}$ and (3) sorting each list of observations twice, first by $t_{o,start}$ and subsequently by $t_{o,end}$, during which the sorting on $t_{o,start}$ is preserved, resulting in a list L_s per segment s .

The batch detection algorithm for segment s takes as input the list L_s , the minimum batch size k_{min} , and the maximum delay parameter γ defining the maximum amount of time allowed between the two successive cases leaving a batch. For a simultaneous batch, $\gamma = 0$. For a disjoint batch, γ should remain small to adhere to the concept of batch processing and should be chosen based on domain knowledge.

For each list, the algorithm will (4) add the first/next observation to a temporary batch and (5) check for each subsequent pair of observations the following constraint:

$$t_{o_{i-1},end} \leq t_{o_i,end} \leq \gamma + t_{o_{i-1},end} \wedge t_{o_i,start} \geq t_{o_{i-1},start} \quad (1)$$

If the constraints holds, we (6a.1) add the next observation to the temporary batch. Otherwise we (6b.1) check whether the size of the temporary batch is $\geq k_{min}$ and if so, create a definitive batch b from it and (6b.2) clear the temporary batch and start over. See [7] for pseudocode.

6 Experimental results

We implemented the batch detection algorithm of Sect. 5 and all measures of Sect. 4 in a Java command-line tool. The tool, available at <https://github.com/multi-dimension-al-process-mining/psm-batchmining>, takes the PSM data [3] as input and returns for each segment two event logs containing batched and non-batched cases and several statistics tables. We applied our implementation on all BPI Challenge event logs except BPIC'16 and BPIC'18 and the Road Traffic Fine Management (RF) log, available at https://data.4tu.nl/repository/collection:event_logs_real, using $k_{min} = 10$ to evaluate noteworthy presence of batching. In the RF log, we found that 15 out of 70 segments have a batch frequency $BF \geq 20\%$ and 8 have a mean batch size $\mu_k \geq 20$. This amounts to 6 out of 4231 with $BF \geq 20$ for the BPIC'11 log, but none with $\mu_k \geq 20$. The BPIC'17 log contains 56 out of 178 segments with $BF \geq 20\%$ and 42 segments with $\mu_k \geq 20$; in BPIC'19, 102 out of 498 segments have $BF \geq 20\%$ and 53 have $\mu_k \geq 20$.

Here, we discuss the RF log and the BPIC'17 log. We show how the algorithm detects batches and provides insights in batching performance from a global (segment) perspective for both logs; for the segment *Create Fine:Send Fine* (RF log) we show how a detailed batch performance analysis gives insights into changes in batching behavior.

We have selected 6 out of 70 segments from the RF log and 1 out of 178 segments from the BPIC'17 log, for which we illustrate the presence of batching and the performance thereof. We have tested various minimum batch sizes to detect clear and distinct batches, resulting in $k_{min} = 20$ for all segments except the third, for which $k_{min} = 30$. Fig. 9 shows for each of the chosen segments S1-S7 the performance spectrum of the detected batches (top) and of all non-batched cases (bottom); for S1 we also show the spectrum for the entire log for illustration. Next, we discuss the quality of the batch identification for S1-S7 in Fig. 9 together with the performance measures for the identified batches which are reported in Tables 1 and 2.

We discover unusual batching behavior in **S1**, where batching is seemingly absent at first glance. However, the projection with batched traces shows distinct larger batches, revealing a meaningful presence of batch processing.

In **S2** we are also able to detect the obvious larger batch among the non-batched traces, while also detecting a few that are significantly smaller. This is confirmed by the high standard deviation for k in Table 1. Additionally, Table 1 also reveals one of the highest $\sigma_{W_o \in b}$ relative to $\mu_{W_o \in b}$ compared to other segments, which indicates very distinct performance in terms of waiting time when comparing the large batch to the smaller ones.

In **S3** we are able to detect a fairly large amount of small batches. Though distinct, the behavior seems non-controlled and most probably indicates high workload on those specific days.

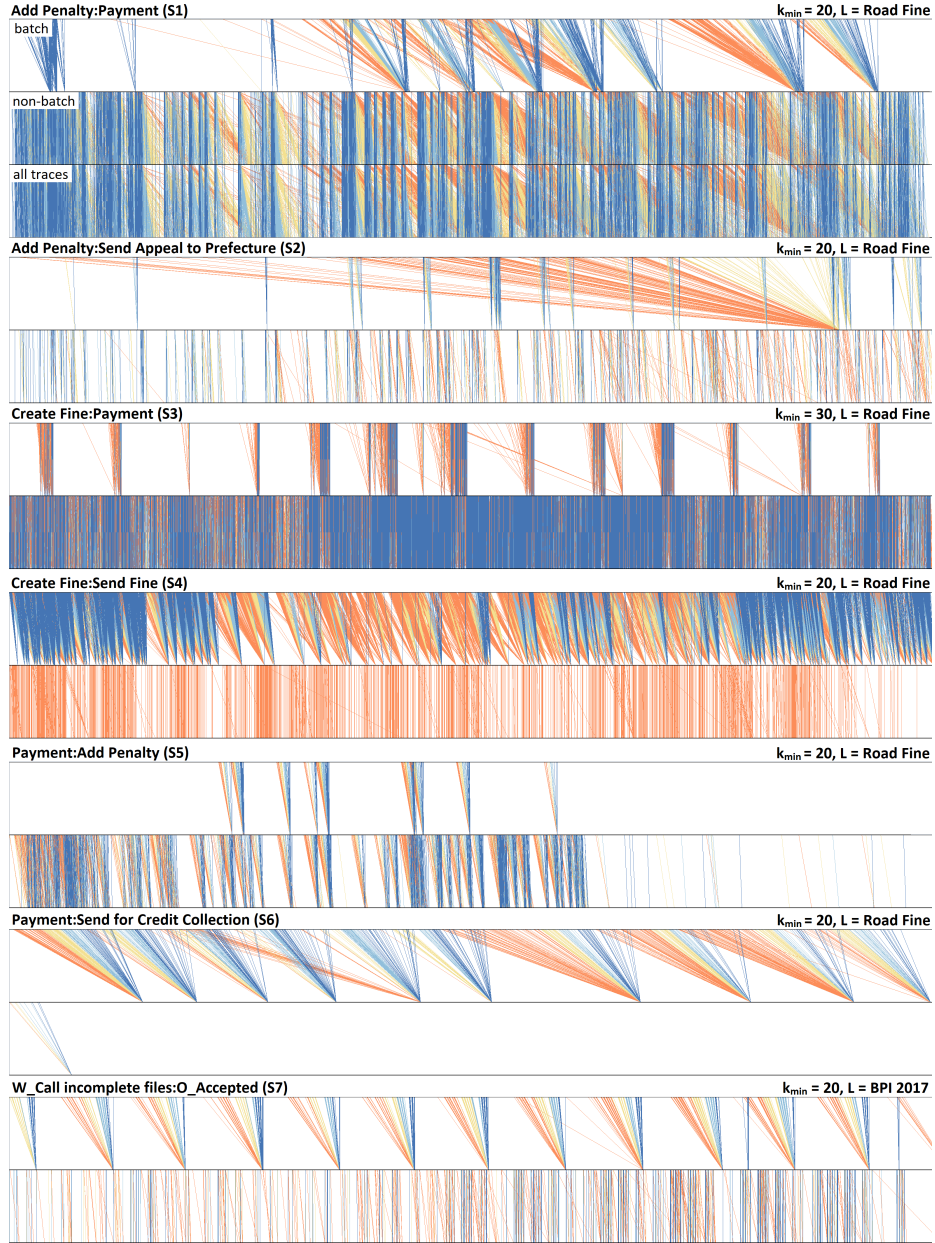


Fig. 9. Detailed performance spectrum of RF and BPIC'17 log for segments S1: *Add Penalty:Payment*, S2: *Add Penalty:Send Appeal to Prefecture*, S3: *Create Fine:Payment*, S4: *Create Fine:Send Fine*, S5: *Payment:Add Penalty*, S6: *Payment:Send for C.C.* and S7: *W_Call incomplete files:O_Accepted*.

Table 1. General performance measurements for S1-S7

Segment	n	BF (%)	m	μ_k	σ_k	μ_{BI}	σ_{BI}	$\mu_{W_o \in b}$	$\sigma_{W_o \in b}$	$\mu_{W_o \notin b}$	$\sigma_{W_o \notin b}$
S1	18621	19.55	104	35.01	25.57	1008.69	2884.14	7047.29	4265.92	3091.11	4070.80
S2	2915	42.54	26	47.69	74.22	4182.72	4233.06	12168.77	20262.79	584.53	910.60
S3	46952	17.67	227	36.55	6.33	472.89	1581.85	241.39	870.23	253.76	481.78
S4	103392	97.15	206	487.61	305.99	569.09	581.72	2161.32	969.51	98.93	589.78
S5	3902	16.27	23	27.61	7.33	1578.50	2807.62	814.83	427.97	792.74	440.05
S6	1538	99.22	10	152.60	41.05	10752.00	3340.28	10697.86	4781.92	6232.00	1340.79
S7	4783	61.61	18	163.72	117.19	518.03	285.79	198.12	140.28	16.61	42.96

In **S4** we discover frequent processing of primarily large batches, which, interestingly, almost all exclusively overlap one another. We also see that non-batched traces are processed almost immediately, which is also revealed by the low $\mu_{W_o \notin b}$, meaning that in terms of performance, batched cases have higher waiting times.

While we do detect a few batches in **S5**, Fig. 9 shows more undetected batches, which is probably a result of the FIFO constraint. Additionally, $\mu_{W_o \in b}$ and $\mu_{W_o \notin b}$ and also $\sigma_{W_o \in b}$ and $\sigma_{W_o \notin b}$ are nearly equal for this segment, indicating similar performance for batched and non-batched traces.

In **S6** we see clear batches that are processed regularly. The non-batch projection shows one small batch is probably not completely recorded in the log, and therefore too small to be detected.

In **S7** (from BPIC'17), we initially did not detect the apparent batches because the FIFO constraint is violated. By a more detailed analysis of the data, we relaxed the batching constraint 1 in our algorithm from FIFO to a 12-hour time-window: cases handled from 0:00 to 11:59 or from 12:00 to 23:59 are part of the same batch. We now detect clear non-overlapping batches, which seem to be processed each month. Unfortunately, the values for μ_{BI} (≈ 22 days) and σ_{BI} (too high), do not correspond. When evaluating batch-level output, we see that three of the larger batches are split in two, due to the batching-window cut-off at 12:00.

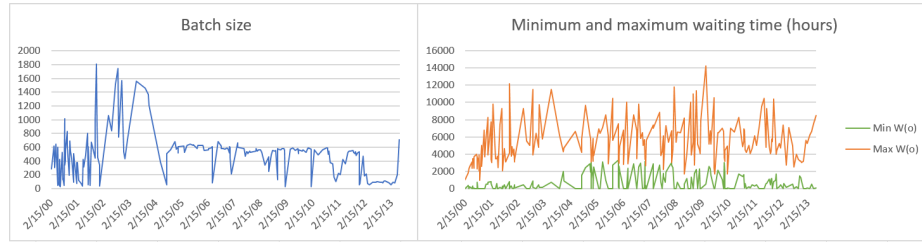
We discover that the three segments with the highest BF (S4, S6 and S7) also have the largest μ_k . While this serves as a good indication for distinct batches, we also found that lower values for these measures do not rule out evident presence of batching. We see that relative to μ_{BI} , σ_{BI} is the lowest for (S6 and S7), which are also the segments for which batching shows up regularly or even periodically on the performance spectrum. For all segments, we find that $\mu_{W_o \in b}$ and $\mu_{W_o \notin b}$ can differ greatly. While this for a part a result of n and BF , we also see that σ relative to μ for these measures varies, indicating that the underlying arrival time distribution differs for batched and non-batched cases. Finally, for segments with batches overlapping (S1-S4 and S6), we see that μ_{IBIA} is always larger than $\mu_{IA \in b}$ and for S5 and S7, this is the other way around. This can be explained by the fact that $IBIA$ is an aggregation of the interarrival time within batches and does not take the arrivals of crossing cases that belong to different batches into account. We think contrasting these measures may help in describing overlapping behavior.

We illustrate a more detailed batch performance analysis by the example of segment *Create Fine:Send Fine* (S4 in Fig. 9). The results are presented in the form of a time series plot in Fig. 10, describing the trend of the measures k , $W_{i,min}$ and $W_{i,max}$. On the left we see that k fluctuates moderately up to nearly the end of 2001, after which it shows excessive increases and decreases until 2005, ranging from a peak of 1806 on

Table 2. Interarrival time measurements for S1-S7

Segment	μ_{IA}	σ_{IA}	$\mu_{IA \in b}$	$\sigma_{IA \in b}$	$\mu_{IA \notin b}$	$\sigma_{IA \notin b}$	μ_{IBIA}	σ_{IBIA}
S1	6.16	31.10	29.68	214.13	7.65	36.78	352.79	861.52
S2	39.12	145.10	89.94	412.52	68.10	198.43	112.69	498.96
S3	2.51	7.99	12.94	203.86	3.05	8.72	48.74	424.87
S4	1.14	5.77	1.17	7.54	39.26	84.84	9.60	104.23
S5	29.62	184.15	57.01	482.74	35.38	200.96	49.61	62.90
S6	71.25	102.70	71.50	103.01	336.00	266.27	115.73	344.86
S7	1.90	7.26	3.06	15.43	4.95	12.91	2.98	13.79

11/9/2001 to a drop of 35 on 12/24/2001. From 2005 onward, the batch size remains predominantly steady and does (for the most part) not exceed 600 cases. On the right we see that $W_{i,max}$ heavily fluctuates between approximately 1000 and 14000 hours ranging the complete time span and a change in trend cannot clearly be observed. However, $W_{i,min}$ does show a small change: it remains close to 0 hours up to 2005, continues to show some higher increases from 2005 to 2010, after which it decreases again. Interestingly, when reviewing the length of the window in which batched cases arrive ($W_{i,max} - W_{i,min}$), we see that due to the trend of $W_{i,min}$, we also observe shorter batching windows between 2005 and 2010, giving insights in batching policy. Detailed batch statistics for this segment are available in [7].

**Fig. 10.** Time series of k and $W_{i,min}$ and $W_{i,max}$ for segment *Create Fine:Send Fine*

7 Conclusion

We have shown that converting event log data into the performance spectrum allows to mine batches from business process event logs without additional information about resources or prior knowledge. The technique is robust to detect batches even in the presence of overlapping with non-batching behavior. By grouping all observations into batches, detailed performance characteristics can be derived which reveals strong performance variations on real-life data. A current limitation is that the technique does not yet incorporate a clear-cut boundary as to when sets of disjoint cases are considered batches or not, nor is it robust to other types of batching behavior, such as non-FIFO. A potential avenue for future work is developing additional detection techniques to cover a greater extent of the performance patterns defined in [4]. Also, detected batches - or even other detected patterns - could be used as features in further performance analysis. Not only could they be used as parameters in queuing networks, complementing [16], they could also prove useful in the area of predictive process monitoring.

References

1. van der Aalst, W.M.P., Adriansyah, A., van Dongen, B.F.: Replaying history on process models for conformance checking and performance analysis. *Wiley Interdisc. Rev.: Data Mining and Knowledge Discovery* **2**(2), 182–192 (2012)
2. Cachon, G., Terwiesch, C.: *Matching Supply with Demand*. McGraw-Hill (2013)
3. Denisov, V., Belkina, E., Fahland, D., van der Aalst, W.M.P.: The performance spectrum miner: Visual analytics for fine-grained performance analysis of processes. In: *BPM 2018 Demos. CEUR Workshop Proceedings*, vol. 2196, pp. 96–100. CEUR-WS.org (2018)
4. Denisov, V., Fahland, D., van der Aalst, W.M.P.: Unbiased, fine-grained description of processes performance from event data. In: *BPM 2018. LNCS*, vol. 11080, pp. 139–157. Springer (2018)
5. Dumas, M., Rosa, M.L., Mendling, J., Reijers, H.A.: *Fundamentals of Business Process Management*. Springer (2013). <https://doi.org/10.1007/978-3-642-33143-5>
6. Henn, S., Koch, S., Wscher, G.: Order batching in order picking warehouses: A survey of solution approaches. In: *Warehousing in the Global Supply Chain*, pp. 105–137. Springer (2012)
7. Klijn, E.L.: Batch pattern detection in the performance spectrum. *Capita selecta research project*, Eindhoven University of Technology (2019). <https://doi.org/10.5281/zenodo.3234102>
8. Martin, N., Solti, A., Mendling, J., Depaire, B., Caris, A.: Mining batch activation rules from event logs. *IEEE Transactions on Services Computing* pp. 1–1 (2019). <https://doi.org/10.1109/TSC.2019.2912163>
9. Martin, N., Swennen, M., Depaire, B., Jans, M., Caris, A., Vanhoof, K.: Batch processing: definition and event log identification. *Proceedings of the 5th International Symposium on Data-driven Process Discovery and Analysis* (2015)
10. Martin, N., Swennen, M., Depaire, B., Jas, M., Caris, A., Vanhoof, K.: Retrieving batch organisation of work insights from event logs. *Decision Support Systems* **100**, 119–128 (2017)
11. Nakatumba, J.: *Resource-aware business process management: analysis and support*. Ph.D. thesis, Eindhoven University of Technology (2013)
12. Pufahl, L., Bazhenova, E., Weske, M.: Evaluating the performance of a batch activity in process models. In: *Lecture Notes in Business Information Processing*. vol. 202, pp. 277–290. Springer (2015)
13. Pufahl, L., Weske, M.: Batch activities in process modeling and execution. In: *Lecture Notes in Computer Science*. vol. 8274, pp. 283–297. Springer (2013)
14. Pufahl, L., Weske, M.: Requirements framework for batch processing in business processes. In: *Lecture Notes in Business Information Processing*. vol. 287, pp. 85–100. Springer (2017)
15. Selvarajah, E., Steiner, G.: Approximation algorithms for the supplier’s supply chain scheduling problem to minimize delivery and inventory holding costs. *Operations Research* **57**(2), 426–438 (3 2009)
16. Senderovich, A., Weidlich, M., Gal, A., Mandelbaum, A.: Queue mining – predicting delays in service processes. In: *Advanced Information Systems Engineering*. pp. 42–57. Springer (2014)
17. Wen, Y., Liu, J., Chen, J.: Mining batch processing workflow models from event logs. *Concurrence and Computation: Practice and Experience* pp. 1928–1942 (2013)